CARDIAC ARRHYTHMIA PREDICTION USING ONE - DIMENSION CONVOLUTIONAL NEURAL NETWORK

A PROJECT – I REPORT

Submitted by

R.NAVISHNA
19RH1A04K0

T.ALEKHYA
19RH1A04M7

T.AISHWARYA
19RH1A04M8

Under the Esteemed Guidance of

Dr.S.Sathish

Professor of ECE

In partial fulfillment of the Academic Requirements for the Degree of

BACHELOR OF TECHNOLOGY

Electronics & Communication Engineering

MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

(Autonomous Institution-UGC, Govt. of India)

Accredited by NBA & NAAC with 'A' Grade

NIRF Indian Ranking, Accepted by MHRD, Govt. of India | Rank Band (6th-25th) by ARIIA, Accepted by MHRD, Govt. of India

Approved by AICTE, Affiliated to JNTUH,ISO9001:2015CertifiedInstitution

Platinum Rated by AICTE-CII Survey, AAAA+ Rated by Digital Learning Magazine, AAA+ Rated by Careers 360, National Ranking-Top 100 Rank band by Outlook Magazine,
2nd Rank by CSR, National Ranking-Top 100 Rank band by Times News Magazine, 141 Rank by India Today-Best Engineering Colleges of India Rankings-2020.

Maisammaguda, Dhulapally, Secunderabad, Kompally-500100.

2022-2023

MALLA REDDY ENGINEERING COLLEGE FOR WOMEN

(Autonomous Institution-UGC, Govt. of India)

Accredited by NBA & NAAC with 'A' Grade

NIRF Indian Ranking, Accepted by MHRD, Govt. of India | Rank Band (6th-25th) by ARIIA, Accepted by MHRD, Govt. of India

Approved by AICTE, Affiliated to JNTUH,ISO9001:2015CertifiedInstitution

Platinum Rated by AICTE-CII Survey, AAAA+ Rated by Digital Learning Magazine, AAA+ Rated by Careers 360, 2nd Rank by CSR, National Ranking-Top 100 Rank band by Outlook Magazine, National Ranking-Top 100 Rank band by Times News Magazine, 141 Rank by India Today-Best Engineering Colleges of India Rankings-2020.

Maisammaguda, Dhulapally, Secunderabad, Kompally-500100.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the Project – I entitled "**CARDIAC ARRHYTHMIA PREDICTION USING ONE–DIMENSION CONVOLUTIONAL NEURAL NETWORK**" is carried out by **T.Alekhya (19RH1A04M7), R.Navishna (19RH1A04K0), T.Aishwarya (19RH1A04M8)** in partial fulfillment for the award of degree of **BACHELOR OF TECHNOLOGY** in Electronics and Communication Engineering, Jawaharlal Nehru Technological University, Hyderabad during the academic year 2022-2023.

Supervisor's Signature	**Head of the Department**
Dr.S.Sathish	**Dr. N.Sreekanth**
Professor of ECE	Professor of ECE

EXTERNAL EXAMINER

CERTIFICATION OF PROJECT COMPLETION

Date: 12-10-2022

This is to certify that the project entitled **"Cardiac Arrhythmia Prediction using One-dimension Convolutional Neural Network"** Submitted in fulfilment of the requirements for the degree in **B. Tech (Electronic communication and Engineering)** From Malla Reddy Engineering College for Women (UGC - AUTONOMOUS), Maisammaguda, Secunderabad, India is a record of bonafide work carried out by

1. **TENTU ALEKHYA** **(19RH1A04M7)**
2. **RAMI REDDY GARI NAVISHNA** **(19RH1A04K0)**
3. **THADISHETTY AISHWARYA** **(19RH1A04M8)**

Under our supervision and guidance. They displayed analytical capability, had innovation approach to solve problems and produced good result.

For SAK INFORMATICS

Managing Director

ACKNOWLEDGEMENT

We feel ourselves honored and privileged to place our warm salutation to our college **Malla Reddy Engineering College for Women** and Department of **Electronics and Communication Engineering** which gave us the opportunity to have expertise in engineering and profound technical knowledge.

We would like to deeply thank our Honorable Minister of Telangana State **Sri.Ch. Malla Reddy Garu,** founder chairman MRGI, the largest cluster of institutions in the state of Telangana for providing us with all the resources in the college to make our project success.

We wish to convey gratitude to our **Principal Dr. Y. Madhavee Latha,** for providing us with the environment and mean to enrich our skills and motivating us in our endeavor and helping us to realize our full potential.

We express our sincere gratitude to **Dr.N.Sreekanth,** Professor and Head of the Department of Electronics and Communication Engineering for inspiring us to take up a project on this subject and successfully guiding us towards its completion.

We would like to thank our Project coordinator **Dr.K.Sudhakar**, Associate Professor for his kind encouragement and overall guidance in viewing this program a good asset with profound gratitude.

We would like to thank our internal guide **Dr.S.Sathish,** Professor and all the Faculty members for their valuable guidance and encouragement towards the completion of our project work.

With Regards and Gratitude,

T.Alekhya(19RH1A04M7)

R.Navishna(19RH1A04K0)

T.Aishwarya(19RH1A04M8)

DECLARATION

We hereby declare that our Project – I entitled **"Cardiac Arrhythmia Prediction Using One-Dimension Convolutional Neural Network"** submitted to **Malla Reddy Engineering College for Women (UGC-Autonomous)** affiliated to **Jawaharlal Nehru Technological University, Hyderabad** for the award of the Degree of Bachelor of Technology in **Electronics and Communication Engineering** is a result of original research work done by us.

It is declared that the Project – I report or any part thereof has not been previously submitted to any University or Institute for the award of Degree.

T.Alekhya (19RH1A04M7)

R.Navishna(19RH1A04K0)

T.Aishwarya (19RH1A04M8)

ABSTRACT

Cardiac arrhythmia is a condition where irregular heart rhythms occur. According to World Health Organization (WHO), about 17 million people in the world die every year due to cardiovascular diseases. This is about 31% of the total deaths globally. According to the statistics of American Heart Association (AHA), one out of every three deaths in US is related to cardiovascular diseases. The deaths due to cardiovascular diseases are more than due to all types of cancer and chronic lower respiratory diseases combined. A 2014 study indicates that approximately 2 to 3% of the people in North American and European countries are affected by atrial fibrillation. A heart rate which is high (above 100 beats per minute in adults) is called tachycardia and a heart rate that is slow (below 60 beats per minute) is called bradycardia. If the beat is too early, then it is called premature contraction. Irregular beat is called fibrillation or flutter. Other than the criteria of heart rate, there are a number of other classifications for cardiac arrhythmia depending upon different types of criteria. Another type of classification is in terms of the site of origin of the irregular heart rate.

Cardiac arrhythmia is a condition where heart beat is irregular. The goal of this paper is to apply deep learning techniques in the diagnosis of cardiac arrhythmia using ECG signals with minimal possible data pre-processing. We employ one-dimension convolutional neural network (1D-CNN), and long short-term memory (LSTM) to automatically detect the abnormality. This work is focused on the design of CNN and LSTM algorithms to predict Arrhythmia diseases with 7 different stages. To train both algorithms, the MIT-BH dataset is used with 7 different disease stages. Further, existing LSTM resulted in low accuracy. So, this work adopted the CNN model for training and testing Arrhythmia disease.

INDEX

TITLE	PAGE NO
CHAPTER 1- INTRODUCTION	**1–3**
1.1 Overview	1-2
1.2 Problem Statement	2-3
CHAPTER 2 - LITERATURE SURVEY	**4-6**
CHAPTER 3- EXISTING METHOD	**7-13**
3.1 ECG Data Acquisition	7-8
3.2 Signal Pre-Processing	8
3.3 ORS Detection	9
3.4 ECG feature extraction using transferred Deep Learning	9-10
3.5 ECG Classification	11-12
3.6 LSTM	12-13
CHAPTER 4 – PROPOSED METHOD	**14-28**
4.1 Proposed Model	14-15
4.2 MIT – BH Dataset	15-16
4.3 Preprocessing	16-19
4.4 Splitting the Dataset	19-20
4.5 PCA Feature extraction	20-22
4.6 1D-CNN Basics	22-26
4.7 Proposed 1D-CNN Classification Model	26-28
CHAPTER 5 – IMPLEMENTATION	**29-40**
5.1 Libraries	29
5.2 Source Code	29-40
CHAPTER 6 – RESULT AND DISCUSSION	
6.1 Modules	41
6.2 Results	42-44

CHAPTER 7- CONCLUSION **45**

Conclusion 45

Future Scope 45

REFERENCES **46-47**

CHAPTER 1
INTRODUCTION

1.1 Overview

Cardiac arrhythmia is a condition where irregular heart rhythms occur. According to World Health Organization (WHO), about 17 million people in the world die every year due to cardiovascular diseases. This is about 31% of the total deaths globally. According to the statistics of American Heart Association (AHA), one out of every three deaths in US is related to cardiovascular diseases. The deaths due to cardiovascular diseases are more than due to all types of cancer and chronic lower respiratory diseases combined. A 2014 study indicates that approximately 2 to 3% of the people in North American and European countries are affected by atrial fibrillation. A heart rate which is high (above 100 beats per minute in adults) is called tachycardia and a heart rate that is slow (below 60 beats per minute) is called bradycardia.

If the beat is too early, then it is called premature contraction. Irregular beat is called fibrillation or flutter. Other than the criteria of heart rate, there are several other classifications for cardiac arrhythmia depending upon different types of criteria. Another type of classification is in terms of the site of origin of the irregular heart rate. Atrial arrhythmias originate in the atrioventricular (AV) node. The AV node is positioned between the atria (each of the two upper cavities of the heart from which blood is passed to the ventricles is referred to as atria) and the ventricles. Atrial fibrillation (AF), atrial flutter, atrial tachycardia, premature atrial contractions, and sinus bradycardia are some examples of atrial arrhythmias. Atrial fibrillation and atrial flutter are examples of arrhythmia which may lead to serious consequences.

In AF, the atrium is contracted in a very fast and irregular manner with the heart's electrical signals originating from a different part of the atria or in the adjacent pulmonary veins instead of sino-atrial (SA) node. The walls of the atria fibrillate (quiver very fast) instead of beating in a normal way, making atria unable to pump blood properly into the ventricles. Stroke and heart failure are two complications to which atrial fibrillation can lead to. Conditions like high blood pressure, overactive thyroid gland, coronary and rheumatic heart diseases can lead to AF. Atrial flutter has similar symptoms and complications as AF. But in atrial flutter, the advancement of electrical signals of the heart through the atria happens in a fast and regular manner instead of the irregular way it happens in AF.

Ventricular arrhythmias are premature rhythms occurring in an ectopic ventricular focus. Ventricular fibrillation, ventricular tachycardia, premature ventricular contractions are some examples of ventricular arrhythmias. Some arrhythmias are symptomless and not at all life threatening. But some symptomless arrhythmias can even lead to serious complications like blood clotting, stroke, heart failure and sudden cardiac death. Arrhythmias occur when the electrical signals to the heart that co-ordinate heartbeat are not working properly. The first step in the diagnosis of this abnormality is the analysis of electrocardiogram (ECG) and the confirmation that the ECG is not indicative of cardiac arrhythmia.

ECG is a bio signal representing the activity of the autonomous nervous system (ANS) controlling heart rhythm. Thus, the electrical activity of the heart is recorded in ECG. It is a non-invasive and efficient tool to study cardiac rhythms and diagnose arrhythmias. The ECG signal is generated because of the following processes. The heartbeat is originated as an electric pulse from the SA node situated in the right atrium of the heart. After contracting both atria, this electric pulse, then activates atrioventricular (AV) node that connects electrically the atria and the ventricles. This is followed by the activation of both ventricles. The complete heart activity is represented in the ECG waveform. Abnormalities in the morphology of ECG waveforms are indicators of cardiac arrhythmias. ECG waveform is analysed to ascertain the risk associated with any type of arrhythmia.

1.2 Problem statement

Extensive research has been done in arrhythmia detection. The below are works in a serious type of arrhythmia called as myocardial infarction (MI) commonly known as heart attack. Data from a single lead ECG was used for MI detection achieving an accuracy of 94.74% [1]. Multiscale eigenspace analysis was carried out on 12 lead ECG data to achieve the same objective with an accuracy of 96% [2]. Analysis of 12 nonlinear parameters extracted from 12 lead ECG data using discrete wavelet transform (DWT) were used to detect MI to achieve an accuracy of 98.8% [3]. Deep learning techniques are now being increasingly employed in this area. The automated detection of normal and MI was conducted with CNN with an accuracy of 95.22% [4]. An accuracy of 84.54% was achieved in the detection of inferior MI in ECG using CNN [5]. Four types of arrhythmias were classified with an accuracy of 99.38% with MIT BIH data set along with another dataset as input [4]. Classification of MIT Arrhythmia database of ECG into normal and abnormal was conducted using artificial neural network (ANN) achieving an accuracy of 96.77% [5].

There are many works of classifying specific types of cardiac arrhythmia with ECG as normal input data. Often these specific cardiac arrhythmia cases addressed in most of the previous research work will be serious arrhythmia types like myocardial infarction. In short, research was conducted into classifying normal ECG and many types of arrhythmias affected ECG. Cardiac arrhythmia, though identified by the irregularity in cardiac rhythm, is due to the anomalies happening in the heart. These anomalies cause anatomical differences in the structure of atria and ventricles, thus producing changes in its activation, depolarization, and repolarisation. These changes are reflected as deviation of ECG waveform from its normal shape and size. Different types of cardiac arrhythmia are caused by unique factors, thus causing unique changes in the morphology of the ECG wave [6], [7], [8].

The objective of this work is to develop an automated method for the diagnosis of cardiac arrhythmia. We perform a two-class classification of the given ECG signal, whether cardiac arrhythmia is present or not. We use ECG recordings from the publically available MIT-BIH arrhythmia database in Physionet. The MIT-BIH arrhythmia database is the first generally available dataset which is widely used for ascertaining the efficiency of cardiac arrhythmia detection algorithms. We employ deep learning-based analysis methods using CNN, CNN-RNN, CNN-LSTM, CNN-GRU. Our work can be an assisting automated tool to cardiologists for the initial screening of people having cardiac arrhythmia.

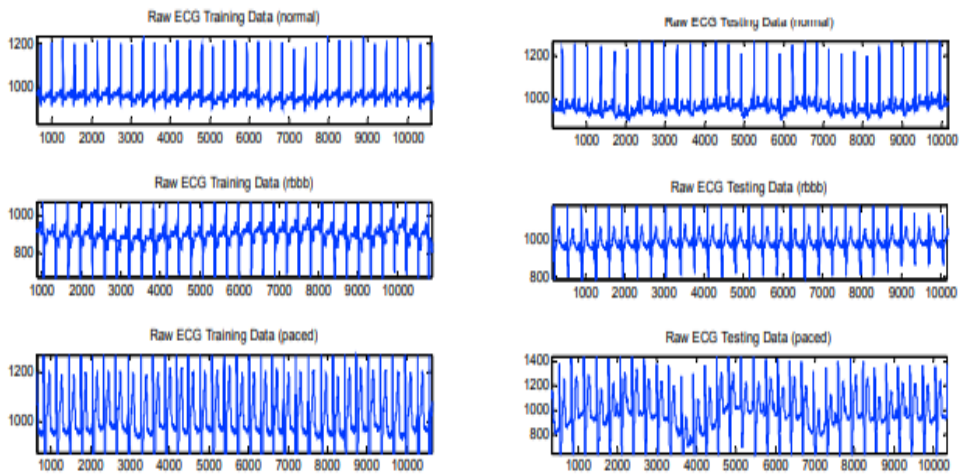


Fig.1: An example of three different ECG recordings.

CHAPTER 2
LITERATURE SURVEY

Jafarnia et al. used two new features i.e., T-wave integral and total integral as extracted feature from one cycle of normal and patient ECG signals to detection and localization of myocardial in-fraction (MI) in left ventricle of heart. And used the T-wave integral because this feature is important impression of T-wave in MI. The second feature in this research is total integral of one ECG cycle, because that the MI affects the morphology of the ECG signal which leads to total integral changes. Also, this work can improve the accuracy of classification by adding more features in this method. A simple method based on using only two features which were extracted from standard ECG is presented and had good accuracy in MI localization.

Sharma et al. studied the multiscale wavelet energies and eigenvalues of multiscale covariance matrices are used as diagnostic features. Support vector machines (SVMs) with both linear and radial basis function (RBF) kernel and K-nearest neighbor are used as classifiers. Datasets, which include healthy control, and various types of MI, such as anterior, anteriolateral, anterioseptal, inferior, inferiolateral, and inferioposterio-lateral, from the PTB diagnostic ECG database are used for evaluation. The results showed that the proposed technique can successfully detect the MI pathologies.

Acharya et al. proposed a novel method of automated detection and localization of MI by using ECG signal analysis. In this study, a total of 200 twelve lead ECG subjects (52 normal and 148 with MI) involving 611,405 beats (125,652 normal beats and 485,753 beats of MI ECG) are segmented from the 12 lead ECG signals. Firstly, ECG signal obtained from 12 ECG leads are subjected to discrete wavelet transform (DWT) up to four levels of decomposition. Then, 12 nonlinear features are extracted from these DWT coefficients. The extracted features are then ranked based on the t value. This proposed method has achieved the highest average accuracy of 98.80%, sensitivity of 99.45% and specificity of 96.27% in classifying normal and MI ECG (two classes), by using 47 features obtained from lead 11 (V5).

Mohammadzadeh and Setarehdan used a neural network classifier to automatic classification of cardiac arrhythmias into five classes. HRV signal is used as the basic signal and linear and nonlinear parameters extracted from it are used to train a neural network classifier. The

proposed approach is tested using the MIT-BIH arrhythmia database and satisfactory results were obtained with an accuracy level of 99.38%.

Vishwa et al. proposed an automated Artificial Neural Network (ANN) based classification system for cardiac arrhythmia using multi-channel ECG recordings. In this study, producing high confident arrhythmia classification results to be applicable in diagnostic decision support systems. The classification performance is evaluated using measures; sensitivity, specificity, classification accuracy, mean squared error (MSE), receiver operating characteristics (ROC) and area under curve (AUC). Experimental results give 96.77% accuracy on MIT-BIH database and 96.21% on database prepared by including NSR database also.

Swapna Etal. discussed the characteristics and different methods (and their measures) of analyting the heart rate variability (HRV) signal, derived from the ECG waveform. The HRV signals are characterised in terms of these measures, then fed into classifiers for grouping into categories (for normal subjects and for disorders such as cardiac disorders and diabetes) for carrying out diagnosis.

Sujadevi etal. explored and employed a deep learning method such as RNN, LSTM and GRU to detect the Atrial Fibrillation (AF) faster in the given electrocardiogram traces. This study used one of the well-known publicly available MIT-BIH Physionet datasets. This is the first time Deep learning has been employed to detect the Atrial Fibrillation in real-time. Based on this work experiments RNN, LSTM and GRU offer the accuracy of 0.950, 1.000 and 1.000 respectively. This methodology does not require any de-noising, other filtering, and preprocessing methods. Results are encouraging enough to begin clinical trials for the real-time detection of AF that will be highly beneficial in the scenarios of ambulatory, intensive care units and for real-time detection of AF for life saving implantable defibrillators.

Pathinarupothi etal. applied a deep learning technique called LSTM-RNN (long short-term memory recurrent neural network) for identification of sleep apnea and its severity based only on instantaneous heart rates. This tested this model on multiple sleep apnea datasets and obtained perfect accuracy. Furthermore, this work has also tested its robustness on an arrhythmia dataset (that is highly probable in mimicking sleep apnea heart rate variability) and found that the model is highly accurate in distinguishing between the two.

Goldberger et al. of cardiovascular and other complex biomedical signals. The resource has 3 interdependent components. PhysioBank is a large and growing archive of well-characterized digital recordings of physiological signals and related data for use by the biomedical research

community. It currently includes databases of multiparameter cardiopulmonary, neural, and other biomedical signals from healthy subjects and from patients with a variety of conditions with major public health implications, including life-threatening arrhythmias, congestive heart failure, sleep apnea, neurological disorders, and aging. PhysioToolkit is a library of open-source software for physiological signal processing and analysis, the detection of physiologically significant events using both classic techniques and novel methods based on statistical physics and nonlinear dynamics, the interactive display and characterization of signals, the creation of new databases, the simulation of physiological and other signals, the quantitative evaluation and comparison of analysis methods, and the analysis of nonstationary processes. PhysioNet is an on-line forum for the dissemination and exchange of recorded biomedical signals and open-source software for analyzing them. It provided facilities for the cooperative analysis of data and the evaluation of proposed new algorithms.

Gers et al. reviewed an illustrative benchmark problem on which standard LSTM outperforms other RNN algorithms. All algorithms (including LSTM) fail to solve a continual version of that problem. LSTM with forget gates, however, easily solves it in an elegant way.

CHAPTER 3
EXSISTING METHOD

In this work, four major steps are applied to solve the ECG pattern recognition and classification problem (Fig. 3.1). These steps include, signal pre-processing, QRS detection, ECG feature extraction using transferred deep learning and ECG signal classification using a conventional LSTM.

Figure 3.1. Block diagram of the Existing classification system.

3.1. ECG Data Acquisition

For this study, the ECG data is obtained from MIT-BIH Arrhythmia database (MIT-BIH ECG database, 2017). MIT-BIH Arrhythmia database is an online database formed using a set of over 4000 long-term ECG Holter recordings. Nearly 60% of these recordings were obtained from patients. The whole database contains 23 records (numbered from 100 to 124, some numbers missing) chosen at random from this set, and 25 records (numbered from 200 to 234, again some numbers missing) selected from the same set to include a variety of rare but clinically important phenomena. Each of these 48 records is slightly over 30 minutes long. All the waveforms present in these recordings are annotated by expert cardiologists. For the proposed system, ECG records 100, 118 and 217 (normal, rbbb and paced beats respectively) are used in the training and records 101, 231 and 107 (normal, rbbb and paced beats respectively) are used in testing process (see Fig.1).

Each of these records has a sample frequency of 360Hz and contains two leads (two signals recorded from different angles on chest). Only one-minute-long sections of each record is extracted obtaining a total of six recordings (3 for training and 3 for testing), each containing 21600 samples and approximately 60-80 waveforms depending on heart rate and class (normal, rbbb or paced). As a final step, one of the channels is removed and only one channel for each recording is used for the rest of the system (channel MLII). In the end of the Data acquisition

part a total of 214 waveforms (as 3 separate recordings each representing a normal, rbbb and paced waveform class) for training and 202 waveforms for testing are prepared ready for next step which is signal pre-processing.

3.2. Signal Pre-processing

Although ECG data obtained from MIT-BIH database is not expected to contain as much disruptive noise as an ECG data obtained directly from a patient, it still contains some noise which requires attention to improve the subsequent steps of the system. Therefore, signal pre-processing step is focused on removing noise from ECG recordings.

As a first step, mean removal is applied to remove the dc noise present in the ECG signals. By subtracting the mean of the ECG recording from every sample point, the unwanted dc component is removed, and the signal baseline amplitude is pulled back to level zero.

Almost all the ECG recordings also contain high and low frequency noise that are imposed by several factors like, the contraction of the muscles, respiratory movements, poor electrode contact and presence of other external devices. To remove high frequency noise, mostly caused by patient's muscle contractions during recording, a 10- point moving average filter which passes low frequencies but attenuates high frequencies is chosen and the signals are filtered. After the removal of high frequency noise, the next step is to remove low frequency noise components. This low frequency noise shows itself as baseline wandering that is caused mostly by the respiration of the patient. To remove this low frequency noise, a derivative based (high pass) filter, which passes high frequencies but attenuates low frequencies, is applied to all ECG recordings.

Power line interference is another type of noise present in the ECG data and is caused by the electricity current flowing in wires and power lines. Power line interference that is present in the ECG signals consists of 60Hz pickup and harmonics. Since frequency of 60Hz overlaps with ECG frequency range (which is generally between 0.05-150 Hz), a filter should be applied to suppress only 60Hz frequency components and its harmonics without disturbing the frequencies around. To achieve this, a comb filter, which is a band-stop filter which attenuates a certain band of frequencies and their harmonics, is used and 60Hz power line interference with its harmonics is removed from the ECG signals.

All the above steps are applied to all training and testing ECG records and filtered ECG signals are obtained ready for the next QRS detection step.

3.3. QRS detection

QRS complex is the most striking waveform within the ECG. Since it reflects the electrical activity within the heart during the ventricular contraction, the time of its occurrence as well as its shape provide valuable information about the current state of the heart. In that sense, QRS detection provides the fundamentals for almost all automated ECG analysis algorithms. Additionally, previous research (Ozbay and Karlik, 2001) proved that in automated ECG classification, focusing on the morphology of the R-T interval (of ECG) is very representative for certain important cardiac conditions, which is also true for rbbb and paced beats. To extract R-T interval from the ECG, Rpeaks should be detected effectively using QRS detection.

A well-known Pan-Tompkins algorithm (Pan and Tompkins, 1985) is applied to carry out the QRS detection. The algorithm includes a series of methods that perform derivative, squaring, integration, adaptive thresholding, and search procedures for the detection of R-peaks of the ECG signal.

3.4. ECG feature extraction using transferred deep learning

As the ECG data in hand is investigated, it can be observed that the features that clearly distinguish each class (normal, rbbb and paced) lies between the R-T interval. Also, it can be easily observed that each member of a class shows same form of pattern in this interval. In this regard, 200 sample points after each R-peak (approximately this number of samples corresponds to the R-T interval with sampling frequency of 360Hz) are extracted (see Fig. 4) and fed as an input to the transferred deep learning-based feature extractor. All other parts of the ECG waveforms are then removed.

AlexNet (Krizhevsky et al., 2012) is a deep learning system, based on a convolutional neural network, which is trained on the 1.2 million high-resolution RGB images of the ImageNet dataset (Deng et al., 2009). This system can classify those images into 1000 different categories with a state-of-the-art performance.

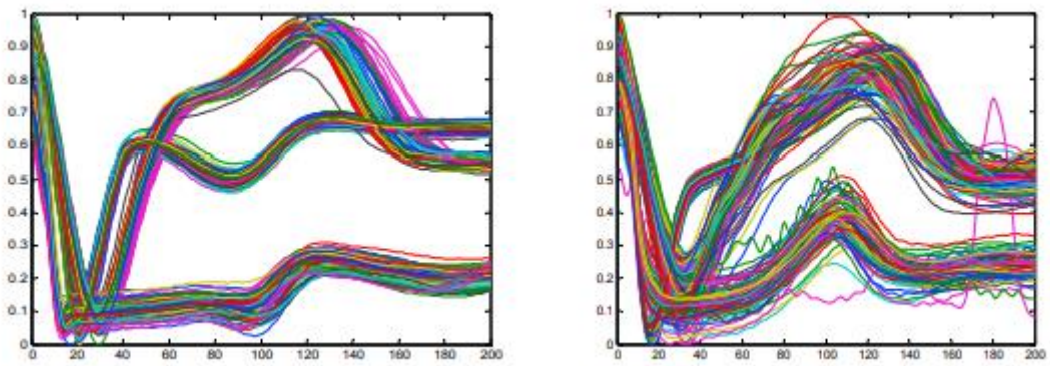


Fig. 3.2: Left: graphical representation of R-T intervals of all training ECG data, right: graphical representation of R-T intervals of all testing ECG data.

The architecture of the AlexNet CNN contains total of eight layers, five convolutional and three fully connected layers, which are trained on the generic images of the ImageNet. As stated in previous research (Yosinski et al. 2014), deeper layers of a deep learning framework trained on a very large, annotated dataset, like ImageNet, can be generic enough to be transferred and implemented for another image classification task. In this regard, for the proposed ECG classification system, the outputs of the deeper layers of the pre-trained AlexNet can be extracted as features for the given ECG inputs. One of the main challenges here is that the AlexNet accepts 256x256 sized RGB images as input where the remaining ECG waveforms after the previously explained steps are discrete signals with 200 sample points. To solve this, all training and testing data (as shown in Fig. 3.2), are converted into 256x256 sized binary images where background and the signal is represented by white and black pixels respectively. Each image representing a single ECG waveform is then reproduced three times to imitate an RGB image.

After the ECG signals are converted into 256x256x3 sized images (total of 214 images for training and 202 image for testing) representing R-T intervals, they are fed into the pre-trained AlexNet and the outputs of the 6th and 7th fully connected layers of the AlexNet are extracted as features representing our R-T segment images for the three different cardiac conditions. Since the fully connected layers of the AlexNet contains 4096 neurons each, this procedure will result in 4096 features for each input ECG waveform that will enable accurate ECG arrhythmia classification. All these obtained features are then converted into feature vectors with 4096 elements (Fv=f1, f2, f3…, f4096) ready for the next classification step.

3.5. ECG classification

Simple conventional multi-layer feed-forward neural networks that have 1 input 1 hidden and 1 output layers are designed to carry out the classification between three different cardiac conditions. These networks are trained using scaled conjugate gradient back-propagation algorithm.

Since the number of features extracted from the 6th and 7th fully connected layers of the AlexNet contains very large number of features (4096 features each) for each ECG R-T segment, we applied principal component analysis (PCA) to reduce the feature vector dimension, decreasing the computational load and risk of over fitting.

After the PCA based feature vector dimension reduction is applied, three different networks are designed and trained. The first network (N-Fc6) takes the output of the 6th fully connected layer of AlexNet as inputs, the second one (N-Fc7) takes the output of the 7th layer, and the third network (N-tst) takes 200 samples of the R-T intervals directly as inputs without any deep learning feature extraction. Additionally, each of these networks are trained using different number of hidden layer neurons and the best performing hidden layer neuron numbers are fixed for the final tests. Output layer neuron number is also fixed as three neurons.

Feature vectors along with their corresponding 214x3 output target vector is fed into the designed networks for training with back propagation algorithm. Training is continued until error goal is achieved or maximum epoch is reached. After the training is finished, networks outputs are compared with output target vector and correct training recognition rates and accuracies are recorded. Correct recognition is counted when the same output neuron shows the maximum value both for actual output and desired output. Accuracy is found by subtracting networks actual output of the neuron that should show the maximum value (that should be classified) from the desired output which is always 1. During training, %15 of the training data and %15 of the testing data is combined and used for validation purposes.

After the training of all three networks are completed, networks are tested using the testing dataset and its corresponding 202x3 output vector, and the performances are recorded for evaluation.

Table 1. Output target vector representations. 214x3 and 202x3 output vectors are formed for training and testing respectively.

Cardiac condition	Corresponding Output Vector
Normal (n)	[1 0 0]
Right Bundle Branch Block (rbbb)	[0 1 0]
Paced Beats (p)	[0 0 1]

3.6 LSTM

LSTM Networks Long Short-Term Memory networks – usually just called "LSTMs" – are a special kind of RNN, capable of learning long-term dependencies. They were introduced by Hochreiter & Schmidhuber and were refined and popularized by many people in following work. They work tremendously well on a large variety of problems and are now widely used. LSTMs are explicitly designed to avoid the long-term dependency problem. All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

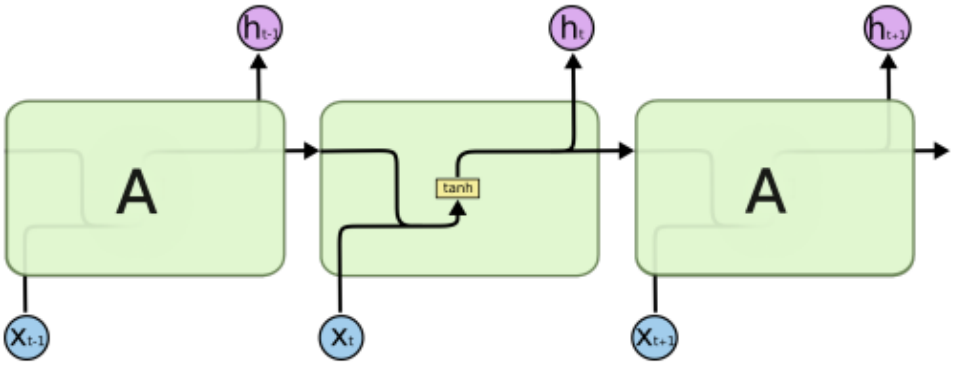


Figure 3.3: LSTM Initialization

LSTMs also have this chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.

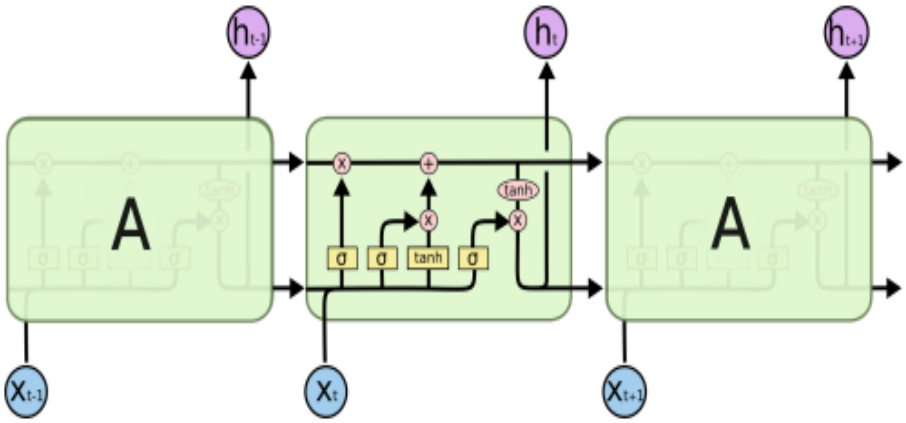


Figure 3.4: LSTM Creation

CHAPTER 4
PROPOSED METHOD

4.1 Proposed Model

Cardiac arrhythmia is a condition where irregular heart rhythms occur. According to World Health Organization (WHO), about 17 million people in the world die every year due to cardiovascular diseases. This is about 31% of the total deaths globally. According to the statistics of American Heart Association (AHA), one out of every three deaths in US is related to cardiovascular diseases. The deaths due to cardiovascular diseases are more than due to all types of cancer and chronic lower respiratory diseases combined. A 2014 study indicates that approximately 2 to 3% of the people in North American and European countries are affected by atrial fibrillation. A heart rate which is high (above 100 beats per minute in adults) is called tachycardia and a heart rate that is slow (below 60 beats per minute) is called bradycardia. If the beat is too early, then it is called premature contraction. Irregular beat is called fibrillation or flutter. Other than the criteria of heart rate, there are a number of other classifications for cardiac arrhythmia depending upon different types of criteria. Another type of classification is in terms of the site of origin of the irregular heart rate.

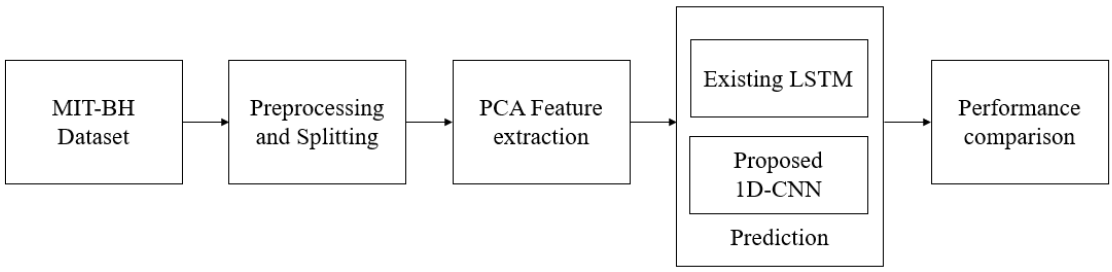


Figure 4.1. Proposed block diagram.

Cardiac arrhythmia is a condition where heart beat is irregular. The goal of this paper is to apply deep learning techniques in the diagnosis of cardiac arrhythmia using ECG signals with minimal possible data pre-processing. We employ deep learning convolutional neural network (1D-CNN), and long short-term memory (LSTM) to automatically detect the abnormality. This work is focused on the design of CNN and LSTM algorithms to predict Arrhythmia diseases with 7 different stages. To train both algorithms, the MIT-BH dataset is

used with 7 different disease stages. Further, existing LSTM resulted in low accuracy. So, this work adopted the CNN model for training and testing Arrhythmia disease.

Figure 4.1 shows the block diagram of proposed. Here, MIT-BH dataset is considered for evaluating the performance of overall system. Initially, the dataset is splitted into 80% for training and 20% for testing. Then, the entire operations are going to be perform on both training and testing datasets. Further, pre-processing operation is carried out to remove the missing symbols and unknown characters, special characters. The pre-processing operation also normalizes the number of rows and columns presented in the dataset. Further, both LSTM and 1D-CNN models are applied to evaluate the prediction of MITBH dataset. So, through this prediction it is going to identify the cardiac arrhythmia presented in overall dataset. Finally, performance comparison is takes place between both LSTM and 1D-CNN models.

4.2 MIT-BH Dataset

The MIT-BIH Arrhythmia Database contains 48 half-hour excerpts of two-channel ambulatory ECG recordings, obtained from 47 subjects studied by the BIH Arrhythmia Laboratory between 1975 and 1979. Twenty-three recordings were chosen at random from a set of 4000 24-hour ambulatory ECG recordings collected from a mixed population of inpatients (about 60%) and outpatients (about 40%) at Boston's Beth Israel Hospital; the remaining 25 recordings were selected from the same set to include less common but clinically significant arrhythmias that would not be well-represented in a small random sample.

The recordings were digitized at 360 samples per second per channel with 11-bit resolution over a 10-mV range. Two or more cardiologists independently annotated each record; disagreements were resolved to obtain the computer-readable reference annotations for each beat (approximately 110,000 annotations in all) included with the database. This directory contains the entire MIT-BIH Arrhythmia Database. About half (25 of 48 complete records, and reference annotation files for all 48 records) of this database has been freely available here since PhysioNet's inception in September 1999.

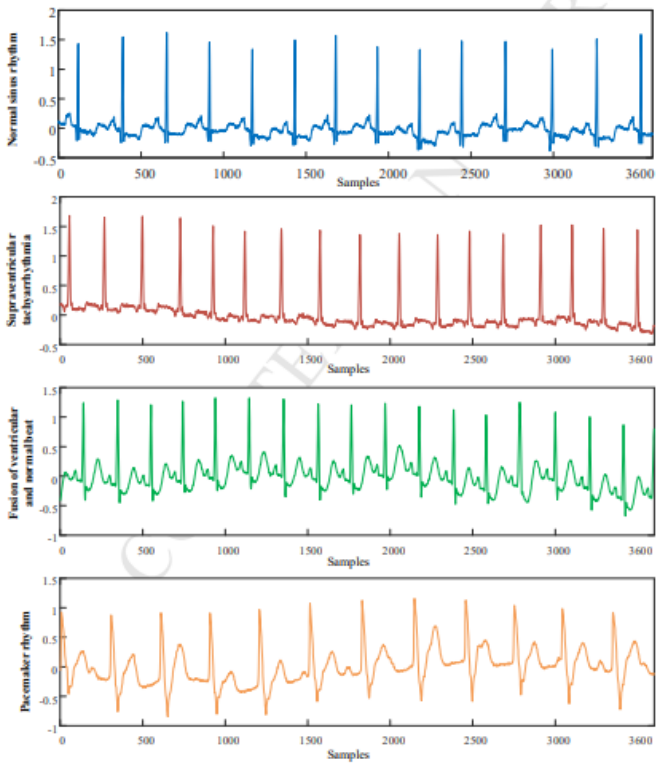


Fig. 4.2: Typical signal samples of different classes.

Finally, the dataset contains ‘Normal heart', 'Ischemic changes (coronary artery disease)', 'Old Anterior Myocardial Infarction', 'Old Inferior Myocardial Infarction', 'Sinus tachycardy', 'Sinus bradycardy', 'Right bundle branch block' as disease classes.

4.3 Preprocessing

Data preprocessing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model. When creating a machine learning project, it is not always a case that we come across the clean and formatted data. And while doing any operation with data, it is mandatory to clean it and put in a formatted way. So, for this, we use data preprocessing task.

Need of Data Preprocessing: A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

- Getting the dataset
- Importing libraries

- Importing datasets
- Finding Missing Data
- Encoding Categorical Data
- Splitting dataset into training and test set
- Feature scaling

Importing Libraries: To perform data preprocessing using Python, we need to import some predefined Python libraries. These libraries are used to perform some specific jobs. There are three specific libraries that we will use for data preprocessing, which are:

Numpy: Numpy Python library is used for including any type of mathematical operation in the code. It is the fundamental package for scientific calculation in Python. It also supports to add large, multidimensional arrays and matrices. So, in Python, we can import it as:

import numpy as nm

Here we have used nm, which is a short name for Numpy, and it will be used in the whole program.

Matplotlib: The second library is matplotlib, which is a Python 2D plotting library, and with this library, we need to import a sub-library pyplot. This library is used to plot any type of charts in Python for the code. It will be imported as below:

import matplotlib.pyplot as mpt

Here we have used mpt as a short name for this library.

Pandas: The last library is the Pandas library, which is one of the most famous Python libraries and used for importing and managing the datasets. It is an open-source data manipulation and analysis library. Here, we have used pd as a short name for this library. Consider the below image:

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

Handling Missing data: The next step of data preprocessing is to handle missing data in the datasets. If our dataset contains some missing data, then it may create a huge problem for our

machine learning model. Hence it is necessary to handle missing values present in the dataset. There are mainly two ways to handle missing data, which are:

- By deleting the particular row: The first way is used to commonly deal with null values. In this way, we just delete the specific row or column which consists of null values. But this way is not so efficient and removing data may lead to loss of information which will not give the accurate output.
- By calculating the mean: In this way, we will calculate the mean of that column or row which contains any missing value and will put it on the place of missing value. This strategy is useful for the features which have numeric data such as age, salary, year, etc.

Encoding Categorical data: Categorical data is data which has some categories such as, in our dataset; there are two categorical variables, Country, and Purchased. Since machine learning model completely works on mathematics and numbers, but if our dataset would have a categorical variable, then it may create trouble while building the model. So, it is necessary to encode these categorical variables into numbers.

Feature Scaling: Feature scaling is the final step of data preprocessing in machine learning. It is a technique to standardize the independent variables of the dataset in a specific range. In feature scaling, we put our variables in the same range and in the same scale so that no variable dominates the other variable. A machine learning model is based on Euclidean distance, and if we do not scale the variable, then it will cause some issue in our machine learning model. Euclidean distance is given as:

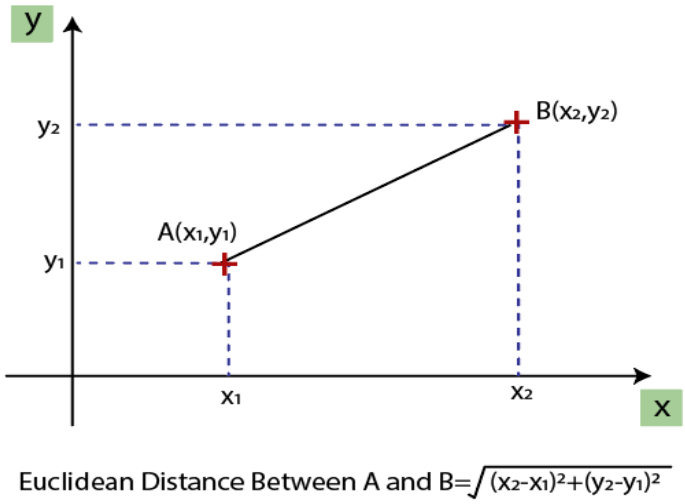
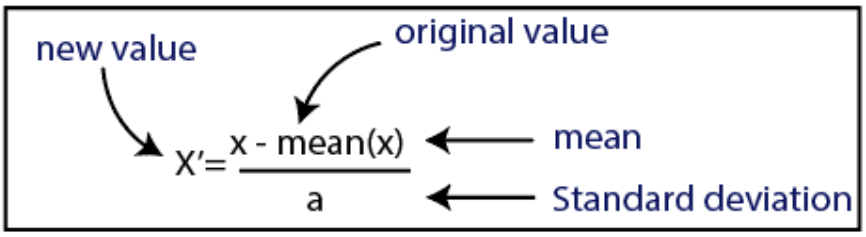


Figure 4.3. Feature scaling

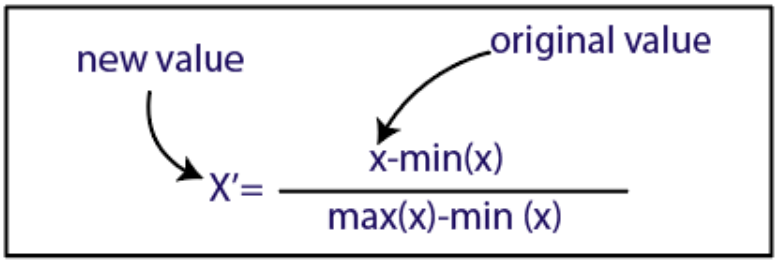
If we compute any two values from age and salary, then salary values will dominate the age values, and it will produce an incorrect result. So to remove this issue, we need to perform feature scaling for machine learning.

There are two ways to perform feature scaling in machine learning:

Standardization



Normalization



4.4 Splitting the Dataset

In machine learning data preprocessing, we divide our dataset into a training set and test set. This is one of the crucial steps of data preprocessing as by doing this, we can enhance the performance of our machine learning model. Suppose if we have given training to our machine learning model by a dataset and we test it by a completely different dataset. Then, it will create difficulties for our model to understand the correlations between the models. If we train our model very well and its training accuracy is also very high, but we provide a new dataset to it, then it will decrease the performance. So we always try to make a machine learning model which performs well with the training set and also with the test dataset. Here, we can define these datasets as:

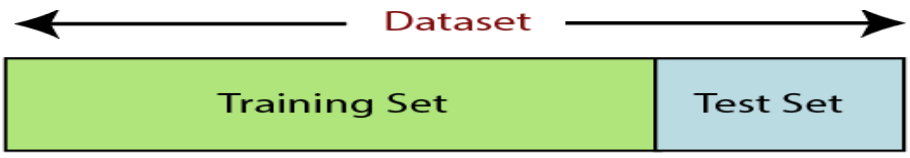


Figure 4.4. Splitting the dataset

Training Set: A subset of dataset to train the machine learning model, and we already know the output.

Test set: A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

For splitting the dataset, we will use the below lines of code:

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.2, random_state=0)
```

Explanation

- In the above code, the first line is used for splitting arrays of the dataset into random train and test subsets.
- In the second line, we have used four variables for our output that are
  - x_train: features for the training data
  - x_test: features for testing data
  - y_train: Dependent variables for training data
  - y_test: Independent variable for testing data
- In train_test_split() function, we have passed four parameters in which first two are for arrays of data, and test_size is for specifying the size of the test set. The test_size maybe .5, .3, or .2, which tells the dividing ratio of training and testing sets.
- The last parameter random_state is used to set a seed for a random generator so that you always get the same result, and the most used value for this is 42.

4.5 PCA Feature extraction

Principal component analysis (PCA) is a popular technique for analyzing large datasets containing a high number of dimensions/features per observation, increasing the interpretability of data while preserving the maximum amount of information, and enabling the visualization of multidimensional data. Formally, PCA is a statistical technique for reducing the dimensionality of a dataset. This is accomplished by linearly transforming the data into a new coordinate system where (most of) the variation in the data can be described with fewer dimensions than the initial data. Many studies use the first two principal components in order to plot the data in two dimensions and to visually identify clusters of closely related data points.

Principal component analysis has applications in many fields such as Population Genetics, Microbiome studies, Atmospheric Science etc.

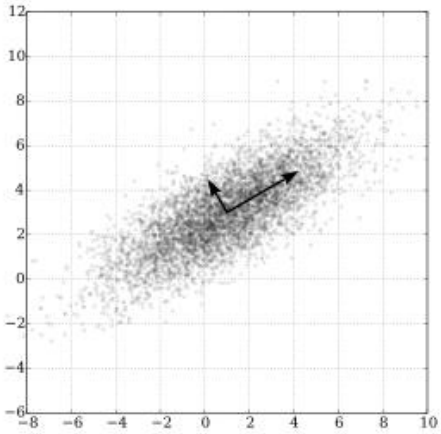


Figure 4.5: PCA of a multivariate Gaussian distribution.

The principal components of a collection of points in a real coordinate space are a sequence of unit vectors, where the vector is the direction of a line that best fits the data while being orthogonal to the first vectors. Here, a best-fitting line is defined as one that minimizes the average squared perpendicular distance from the points to the line. These directions constitute an orthonormal basis in which different individual dimensions of the data are linearly uncorrelated. PCA is the process of computing the principal components and using them to perform a change of basis on the data, sometimes using only the first few principal components and ignoring the rest.

In data analysis, the first principal component of a set of variables, presumed to be jointly normally distributed, is the derived variable formed as a linear combination of the original variables that explains the most variance. The second principal component explains the most variance in what is left once the effect of the first component is removed, and we may proceed through iterations until all the variance is explained. PCA is most used when many of the variables are highly correlated with each other, and it is desirable to reduce their number to an independent set.

PCA is used in exploratory data analysis and for making predictive models. It is commonly used for dimensionality reduction by projecting each data point onto only the first few principal components to obtain lower-dimensional data while preserving as much of the data's variation

as possible. The first principal component can equivalently be defined as a direction that maximizes the variance of the projected data. The principal component can be taken as a direction orthogonal to the first principal components that maximizes the variance of the projected data.

For either objective, it can be shown that the principal components are eigenvectors of the data's covariance matrix. Thus, the principal components are often computed by eigen decomposition of the data covariance matrix or singular value decomposition of the data matrix. PCA is the simplest of the true eigenvector-based multivariate analyses and is closely related to factor analysis. Factor analysis typically incorporates more domain specific assumptions about the underlying structure and solves eigenvectors of a slightly different matrix. PCA is also related to canonical correlation analysis (CCA). CCA defines coordinate systems that optimally describe the cross-covariance between two datasets while PCA defines a new orthogonal coordinate system that optimally describes variance in a single dataset. Robust and L1-norm-based variants of standard PCA have also been proposed.

4.6 1D-CNN Basics

According to the facts, training and testing of 1D-CNN involves in allowing every source image via a succession of convolution layers by a kernel or filter, rectified linear unit (ReLU), max pooling, fully connected layer and utilize SoftMax layer with classification layer to categorize the objects with probabilistic values ranging from [0,1]. Figure 1 discloses the architecture of 1D-CNN that is utilized in proposed methodology for CBIR system for enhanced feature representation of word image over conventional retrieval systems.

Convolution layer as depicted in Figure 4.6 is the primary layer to extract the features from a source image and maintains the relationship between pixels by learning the features of image by employing tiny blocks of source data. It's a mathematical function which considers two inputs like source image $I(x, y, d)$ where x and y denotes the spatial coordinates i.e., number of rows and columns. d is denoted as dimension of an image (here $d = 3$, since the source image is RGB) and a filter or kernel with similar size of input image and can be denoted as $F(k_x, k_y, d)$.

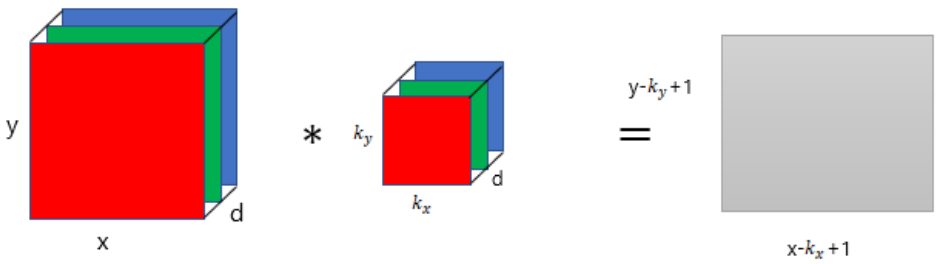


Fig. 4.6: Representation of convolution layer process

The output obtained from convolution process of input image and filter has a size of $C\left((x-k_x+1),(y-k_y+1),1\right)$, which is referred as feature map. An example of convolution procedure is demonstrated in Figure 4.7. Let us assume an input image with a size of 5×5 and the filter having the size of 3×3.

1	1	1	0	0
0	0	1	1	1
1	1	0	0	1
0	0	0	1	1
1	1	1	0	0

5x5 image

*

1	0	1
0	1	0
1	0	1

3x3 kernel

(a)

1	1	1	0	0
0	0	1	1	1
1	1	0	0	1
0	0	0	1	1
1	1	1	0	0

5x5 image

*

1	0	1
0	1	0
1	0	1

3x3 kernel

=

3	3	3
2	2	3
3	2	3

Feature map

(b)

Fig. 4.7: Example of convolution layer process (a) an image with size $\mathbf{5 \times 5}$ is convolving with $\mathbf{3 \times 3}$ kernel (b) Convolved feature map

ReLU layer

Networks those utilizes the rectifier operation for the hidden layers are cited as rectified linear unit (ReLU). This ReLU function $\mathcal{G}(\cdot)$ is a simple computation that returns the value given as input directly if the value of input is greater than zero else returns zero. This can be

represented as mathematically using the function $max(\cdot)$ over the set of 0 and the input x as follows:

$$\mathcal{G}(x) = \max\{0, x\}$$

Max pooing layer

This layer mitigates the number of parameters when there are larger size images. This can be called as subsampling or down sampling that mitigates the dimensionality of every feature map by preserving the important information. Max pooling considers the maximum element form the rectified feature map.

SoftMax classifier

Generally, as seen in the above picture SoftMax function is added at the end of the output since it is the place where the nodes are meet finally and thus, they can be classified.

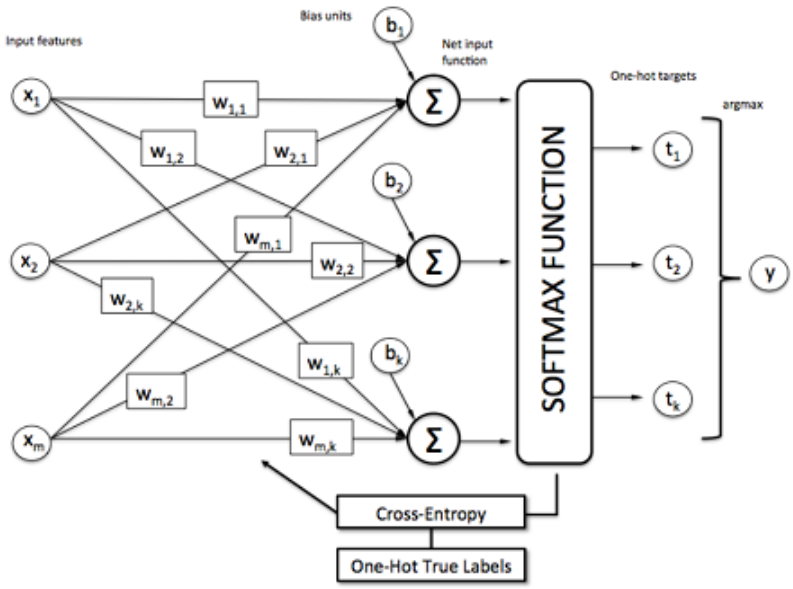


Figure 4.8: SoftMax classifier.

Here, X is the input of all the models and the layers between X and Y are the hidden layers and the data is passed from X to all the layers and Received by Y. Suppose, we have 10 classes, and we predict for which class the given input belongs to. So, for this what we do is allot each class with a particular predicted output. Which means that we have 10 outputs corresponding to 10 different class and predict the class by the highest probability it has.

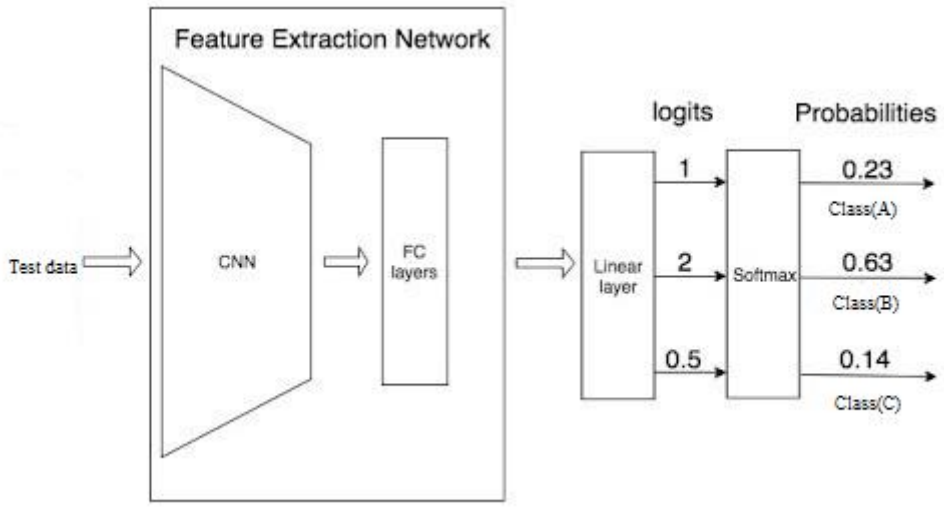


Figure 4.9. Probabilities extraction using SoftMax classifier

In the above, a picture is given, and we must predict what is the object that is present in the picture. In the normal case, we predict whether the crop is A. But in this case, we must predict what is the object that is present in the picture. This is the place where softmax comes in handy. As the model is already trained on some data. So, as soon as the picture is given, the model processes the pictures, send it to the hidden layers and then finally send to softmax for classifying the picture. The softmax uses a One-Hot encoding Technique to calculate the cross-entropy loss and get the max. One-Hot Encoding is the technique that is used to categorize the data. In the previous example, if softmax predicts that the object is class A then the One-Hot Encoding for: -

Class A will be [1 0 0]

Class B will be [0 1 0]

Class C will be [0 0 1]

From the diagram, we see that the predictions are occurred. But generally, we don't know the predictions. But the machine must choose the correct predicted object. So, for machine to identify an object correctly, it uses a function called cross-entropy function. So, we choose more similar value by using the below cross-entropy formula.

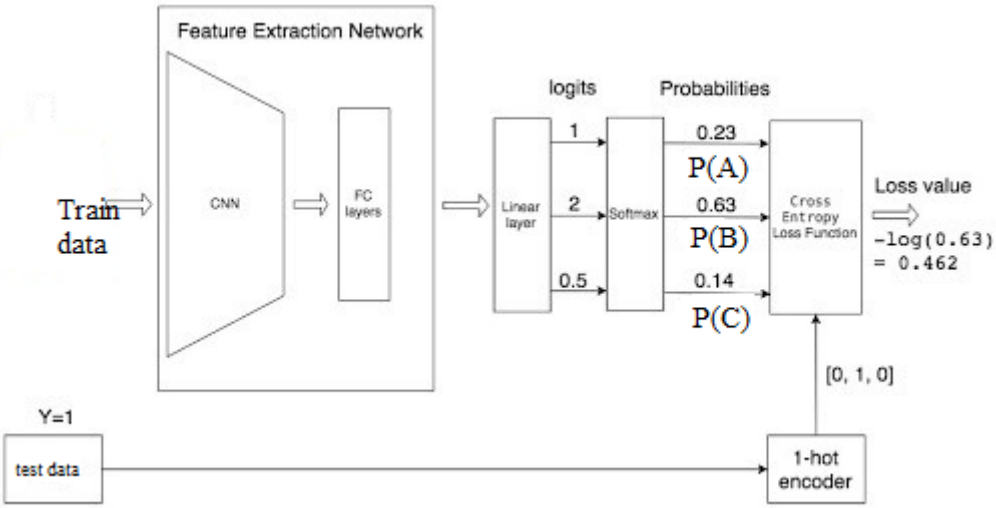


Figure 4.10. Test probabilities extraction using SoftMax classifier

In the above example we see that 0.462 is the loss of the function for class specific classifier. In the same way, we find loss for remaining classifiers. The lowest the loss function, the better the prediction is.

The mathematical representation for loss function can be represented as: -

$$LOSS = np.sum(-Y * np.log(Y_pred))$$

4.7 Proposed 1D-CNN Classification Model

A 6-layer deep convolutional network was designed for the classification of ECG signals according to cardiac arrhythmia. This deep network model provides automatic classification of input fragments through an end-to-end structure without the need for any hand-crafted feature extraction or selection steps. The structure of the deep network model consists of the classical CNN layers, but the structure of 1D-CNN is predominant. In 1D convolution layers, feature maps that are representations of ECG fragments are subjected to convolution processing with weights of various sizes.

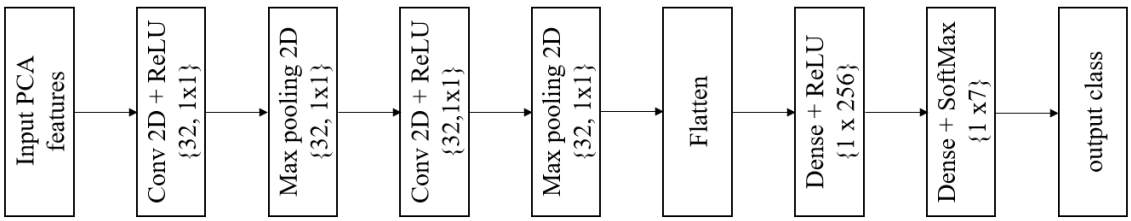


Figure 4.12: Block diagram of the proposed 1D-CNN model.

Table.1: Layers description.

Layer Names	No. of filters	Kernel size
Conv 2D +ReLU	32	1x 1
Max pooling 2D	-	1 x 1
Conv 2D+ReLU	32	1 x 1
Max pooling 2D	-	1 x 1
Flatten	-	-
Dense +ReLU		1 x 256
Dense + SoftMax		1 x 7

In the first layer of the model, 1D convolution is performed with 128 weight vectors on the input ECG signals. The activation outputs of this layer are normalized using batch normalization layer for each batch. In the 1D max pooling layer, new feature maps are generated by taking the maximum values in the region specified on the feature maps obtained from the previous layers. This layer reduces the size of feature maps from the previous layer according to the region size. The reducing feature map sizes is an important step in reducing the computational cost of deep learning structures. For this purpose, different methods such as average values are used instead of maximum values in 1D Max layer. In the fourth layer, the convolution process is repeated on the input feature maps with 32×7- size weights. By performing batch normalization process again, the feature maps whose region width is set to two are reduced by half using pooling method on the 6th layer. These operations are repeated in the next 1D convolution and 1D max pooling layers.

The proposed deep network has a flattened layer on the 14th layer so that the feature maps obtained from the 13th layer can be transformed to the appropriate size as an input to the subsequent layers of the network. This layer transforms multidimensional input feature vectors into one-dimensional output data. The features obtained from the flattened layer are fed to a

dense-connected neural network layer of 512 units. In the last layer of the network, there is a layer of softmax which is the unit of the number of the output classes. Using the softmax layer, the prediction of the class to which the input data belongs is realized. In addition to all of these, some layers have a dropout parameter to prevent the overfitting during the learning phase. After developing the model, the layer numbers, types and parameters of the deep algorithm are changed by brute force technique and the performances of the validation sets are observed. Our developed 6-layer model yielded the highest classification results for long duration ECG signals.

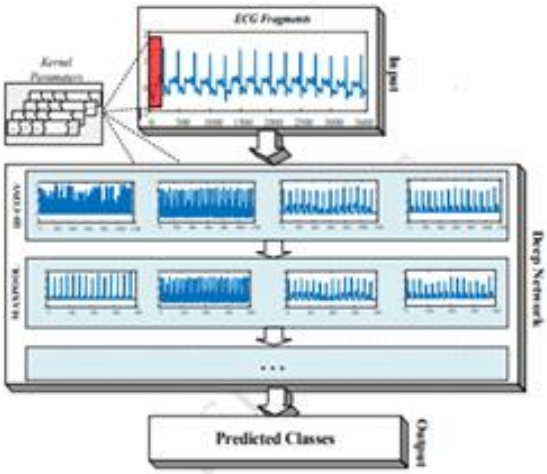


Fig. 4.13: Illustration of 1D convolution and max pooling processes on the ECG fragment signals.

Advantage of proposed system: This research showed many advantages, including its ability to help clinicians reliably make ECG recording-related clinical decisions. Moreover, it was intended to be as simple as possible while delivering the most significant performance. The described method is straightforward for health professionals and does not involve signal modification or feature extraction. Additionally, this research focused only on one kind of CVD, namely, arrhythmia, whereas the manifestations of cardiac disease are often complex and varied. As a result, more types of ECG data will need to be added to broaden the scope of the planned network.

CHAPTER 5
IMPLEMENTATION

5.1 Libraries

- pip install numpy==1.19.2
- pip install pandas==0.25.3
- pip install matplotlib==3.1.1
- pip install keras==2.3.1
- pip install tensorflow==1.14.0
- pip install h5py==2.10.0
- pip install protobuf==3.16.0
- pip install scikit-learn==0.22.2.post1
- pip install seaborn==0.10.1

5.2 Source Code

```
from tkinter import *

import tkinter

from tkinter import filedialog

import matplotlib.pyplot as plt

from tkinter.filedialog import askopenfilename

import numpy as np

import pandas as pd

from sklearn.metrics import accuracy_score

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix

import seaborn as sns

from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
import os
import webbrowser
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import LSTM
from keras.utils.np_utils import to_categorical
from keras.models import model_from_json
import pickle
from keras.layers import  MaxPooling2D
from keras.layers import  Activation, Flatten
from keras.layers import Convolution2D
main = tkinter.Tk()
main.title("Automated Detection of Cardiac Arrhythmia using Recurrent Neural Network")
main.geometry("1200x1200")
global X_train, X_test, y_train, y_test, pca
global model, dataset
global filename
```

```
global X, Y

accuracy = []

precision = []

recall = []

fscore = []

sensitivity = []

specificity = []

labels = ['Normal heart', 'Ischemic changes (Coronary Artery Disease)', 'Old Anterior
Myocardial Infarction',

        'Old Inferior Myocardial Infarction', 'Sinus tachycardy', 'Sinus bradycardy', 'Right
bundle branch block']

def uploadDataset():

    global filename, dataset

    text.delete('1.0', END)

    filename = filedialog.askopenfilename(initialdir="Dataset")

    text.insert(END,str(filename)+" Dataset Loaded\n\n")

    pathlabel.config(text=str(filename)+" Dataset Loaded")

    dataset = pd.read_csv(filename)

    text.insert(END,str(dataset.head()))

    label = dataset.groupby('279').size()

    label.plot(kind="bar")

    plt.show()

def preprocessDataset():

    global X, Y, dataset, pca

    global X_train, X_test, y_train, y_test

    text.delete('1.0', END)
```

```
le = LabelEncoder()
dataset.fillna(0, inplace = True)
dataset["279"] = pd.Series(le.fit_transform(dataset["279"].astype(str)))
temp = dataset.values
X = temp[:,0:temp.shape[1]-1]
Y = temp[:,temp.shape[1]-1]
indices = np.arange(X.shape[0])
np.random.shuffle(indices)
X = X[indices]
Y = Y[indices]
print(Y)
X = normalize(X)
pca = PCA(n_components = 40)
X = pca.fit_transform(X)
Y = to_categorical(Y)
XX = np.reshape(X, (X.shape[0], X.shape[1], 1))
X_train, X_test, y_train, y_test = train_test_split(XX, Y, test_size = 0.2)
text.insert(END,"Total records found in dataset : "+str(X.shape[0])+"\n\n")
text.insert(END,"Different diseases found in dataset\n\n")
text.insert(END,str(labels)+"\n\n")
text.insert(END,"Dataset Train & Test Split Details\n\n")
text.insert(END,"Total records used to train LSTM & CNN : "+str(X_train.shape[0])+"\n")
text.insert(END,"Total records used to test LSTM & CNN  : "+str(X_test.shape[0])+"\n")
def calculateMetrics(algorithm, predict, y_test):
a = accuracy_score(y_test,predict)*100
```

```
p = precision_score(y_test, predict,average='macro') * 100

r = recall_score(y_test, predict,average='macro') * 100

f = f1_score(y_test, predict,average='macro') * 100

accuracy.append(a)

precision.append(p)

recall.append(r)

fscore.append(f)

text.insert(END,algorithm+" Accuracy  :  "+str(a)+"\n")

text.insert(END,algorithm+" Precision : "+str(p)+"\n")

text.insert(END,algorithm+" Recall    : "+str(r)+"\n")

text.insert(END,algorithm+" FScore    : "+str(f)+"\n")

conf_matrix = confusion_matrix(y_test, predict)

total = sum(sum(conf_matrix))

se = conf_matrix[0,0]/(conf_matrix[0,0]+conf_matrix[0,1])

text.insert(END,algorithm+' Sensitivity : '+str(se)+"\n")

sp = conf_matrix[1,1]/(conf_matrix[1,0]+conf_matrix[1,1])

text.insert(END,algorithm+' Specificity : '+str(sp)+"\n\n")

sensitivity.append(se)

specificity.append(sp)

text.update_idletasks()

plt.figure(figsize =(6, 6))

ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True,
cmap="viridis" ,fmt ="g");

ax.set_ylim([0,len(labels)])

plt.title(algorithm+" Confusion matrix")
```

```
    plt.ylabel('True class')
    plt.xlabel('Predicted class')
    plt.show()
def runLSTM():
    global X_train, X_test, y_train, y_test
    global accuracy, precision, recall, fscore, sensitivity, specificity
    text.delete('1.0', END)
    accuracy.clear()
    precision.clear()
    recall.clear()
    fscore.clear()
    sensitivity.clear()
    specificity.clear()
    if os.path.exists('model/lstm_model.json'):
        with open('model/lstm_model.json', "r") as json_file:
            loaded_model_json = json_file.read()
            lstm = model_from_json(loaded_model_json)
        json_file.close()
        lstm.load_weights("model/lstm_model_weights.h5")
        lstm._make_predict_function()
    else:
        lstm_model = Sequential()#defining deep learning sequential object
        #adding LSTM layer with 100 filters to filter given input X train data to select relevant
features
        lstm_model.add(LSTM(100,input_shape=(X_train.shape[1], X_train.shape[2])))
```

```
        #adding dropout layer to remove irrelevant features

        lstm_model.add(Dropout(0.2))

        #adding another layer

        lstm_model.add(Dense(100, activation='relu'))

        #defining output layer for prediction

        lstm_model.add(Dense(y_train.shape[1], activation='softmax'))

        #compile LSTM model

        lstm_model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

        #start training model on train data and perform validation on test data

        hist = lstm_model.fit(X_train, y_train, epochs=100, batch_size=16,
validation_data=(X_test, y_test))

        #save model weight for future used

        lstm_model.save_weights('model/lstm_model_weights.h5')

        model_json = lstm_model.to_json()

        with open("model/lstm_model.json", "w") as json_file:

            json_file.write(model_json)

        json_file.close()

        f = open('model/lstm_history.pckl', 'wb')

        pickle.dump(hist.history, f)

        f.close()

    print(lstm.summary())

    predict = lstm.predict(X_test)

    predict = np.argmax(predict, axis=1)

    testY = np.argmax(y_test, axis=1)

    calculateMetrics("LSTM", predict, testY)
```

```
def runCNN():
    global X, Y
    XX = np.reshape(X, (X.shape[0], X.shape[1], 1, 1))
    X_train, X_test, y_train, y_test = train_test_split(XX, Y, test_size = 0.2)
    if os.path.exists('model/cnn_model.json'):
        with open('model/cnn_model.json', "r") as json_file:
            loaded_model_json = json_file.read()
            cnn = model_from_json(loaded_model_json)
        json_file.close()
        cnn.load_weights("model/cnn_model_weights.h5")
        cnn._make_predict_function()
    else:
        cnn = Sequential()
        cnn.add(Convolution2D(32, 1, 1, input_shape = (X_train.shape[1], X_train.shape[2],
X_train.shape[3]), activation = 'relu'))
        cnn.add(MaxPooling2D(pool_size = (1, 1)))
        cnn.add(Convolution2D(32, 1, 1, activation = 'relu'))
        cnn.add(MaxPooling2D(pool_size = (1, 1)))
        cnn.add(Flatten())
        cnn.add(Dense(output_dim = 256, activation = 'relu'))
        cnn.add(Dense(output_dim = y_train.shape[1], activation = 'softmax'))
        cnn.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])
        hist = cnn.fit(X_train, y_train, batch_size=16, epochs=100, shuffle=True, verbose=2,
validation_data=(X_test, y_test))
        cnn.save_weights('model/cnn_model_weights.h5')
```

```
        model_json = cnn.to_json()
        with open("model/cnn_model.json", "w") as json_file:
            json_file.write(model_json)
        json_file.close()
        f = open('model/cnn_history.pckl', 'wb')
        pickle.dump(hist.history, f)
        f.close()
    print(cnn.summary())
    predict = cnn.predict(X_test)
    predict = np.argmax(predict, axis=1)
    testY = np.argmax(y_test, axis=1)
    calculateMetrics("CNN", predict, testY)
def graph():
    f = open('model/lstm_history.pckl', 'rb')
    data = pickle.load(f)
    f.close()
    lstm_accuracy = data['accuracy']
    lstm_loss = data['loss']
    f = open('model/cnn_history.pckl', 'rb')
    data = pickle.load(f)
    f.close()
    cnn_accuracy = data['accuracy']
    cnn_loss = data['loss']
```

```
    plt.figure(figsize=(10,6))

    plt.grid(True)

    plt.xlabel('EPOCH')

    plt.ylabel('Accuracy/Error Rate')

    plt.plot(lstm_accuracy, 'ro-', color = 'green')

    plt.plot(lstm_loss, 'ro-', color = 'blue')

    plt.plot(cnn_accuracy, 'ro-', color = 'orange')

    plt.plot(cnn_loss, 'ro-', color = 'red')

    plt.legend(['LSTM Accuracy', 'LSTM Loss','CNN Accuracy','CNN Loss'], loc='upper left')

    plt.title('LSTM Vs CNN Training Accuracy & Loss Graph')

    plt.show()

def performanceTable():

    output = '<table border=1 align=center>'

    output+= '<tr><th>Dataset Name</th><th>Algorithm
Name</th><th>Accuracy</th><th>Precision</th><th>Recall</th><th>FSCORE</th><th>S
ensitivity</th><th>Specificity</th></tr>'

    output+='<tr><td>MIT-BH
Dataset</td><td>LSTM</td><td>'+str(accuracy[0])+'</td><td>'+str(precision[0])+'</td><td
>'+str(recall[0])+'</td><td>'+str(fscore[0])+'</td><td>'+str(sensitivity[0])+'</td><td>'+str(sp
ecificity[0])+'</td></tr>'

    output+='<tr><td>MIT-BH
Dataset</td><td>CNN</td><td>'+str(accuracy[1])+'</td><td>'+str(precision[1])+'</td><td>'
+str(recall[1])+'</td><td>'+str(fscore[1])+'</td><td>'+str(sensitivity[1])+'</td><td>'+str(spe
cificity[1])+'</td></tr>'

    output+='</table></body></html>'

    f = open("output.html", "w")

    f.write(output)

    f.close()

    webbrowser.open("output.html",new=1)
```

```
def close():
    main.destroy()
font = ('times', 14, 'bold')
title = Label(main, text='Automated Detection of Cardiac Arrhythmia using Recurrent Neural
Network')
title.config(bg='DarkGoldenrod1', fg='black')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=5,y=5)
font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload Arrhythmia Dataset", command=uploadDataset)
uploadButton.place(x=50,y=100)
uploadButton.config(font=font1)
pathlabel = Label(main)
pathlabel.config(bg='brown', fg='white')
pathlabel.config(font=font1)
pathlabel.place(x=560,y=100)
preprocessButton = Button(main, text="Preprocess Dataset", command=preprocessDataset)
preprocessButton.place(x=50,y=150)
preprocessButton.config(font=font1)
lstmButton = Button(main, text="Run LSTM Algorithm", command=runLSTM)
lstmButton.place(x=50,y=200)
lstmButton.config(font=font1)
cnnButton = Button(main, text="Run CNN Algorithm", command=runCNN)
```

```
cnnButton.place(x=50,y=250)
cnnButton.config(font=font1)
graphButton = Button(main, text="LSTM & CNN Training Graph", command=graph)
graphButton.place(x=50,y=300)
graphButton.config(font=font1)
ptButton = Button(main, text="Performance Table", command=performanceTable)
ptButton.place(x=50,y=350)
ptButton.config(font=font1)
exitButton = Button(main, text="Exit", command=close)
exitButton.place(x=50,y=400)
exitButton.config(font=font1)
font1 = ('times', 12, 'bold')
text=Text(main,height=25,width=100)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=400,y=150)
text.config(font=font1)
main.config(bg='LightSteelBlue1')
main.mainloop()
```

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Modules

To implement this project to designed following modules

1) Upload Arrhythmia Dataset: using this module will upload dataset to application. Finally, the dataset contains ‘Normal heart', 'Ischemic changes (coronary artery disease)', 'Old Anterior Myocardial Infarction', 'Old Inferior Myocardial Infarction', 'Sinus tachycardy', 'Sinus bradycardy', 'Right bundle branch block' as disease classes.

2) Pre-process Dataset: this module will read all dataset values and then replace missing values with MEAN and then normalize training values and then selected important features from dataset by applying PCA algorithm. Dataset contains more than 270 columns and all these columns are not required so PCA selected relevant features from dataset. After features selection dataset splitted into train and test where application using 80% dataset for training and 20% for testing.

3) Run LSTM Algorithm: the 80% training data is applied input to LSTM to trained a model and then model will be applied on 20% test data to perform prediction and then calculate accuracy.

4) Run CNN Algorithm: the 80% training data is applied input to CNN to trained a model and then model will be applied on 20% test data to perform prediction and then calculate accuracy.

5) LSTM & CNN Training Graph: this module plots CNN and LSTM training graph

6) Performance Table: this module displays both algorithms performance in tabular format.

6.2 Results

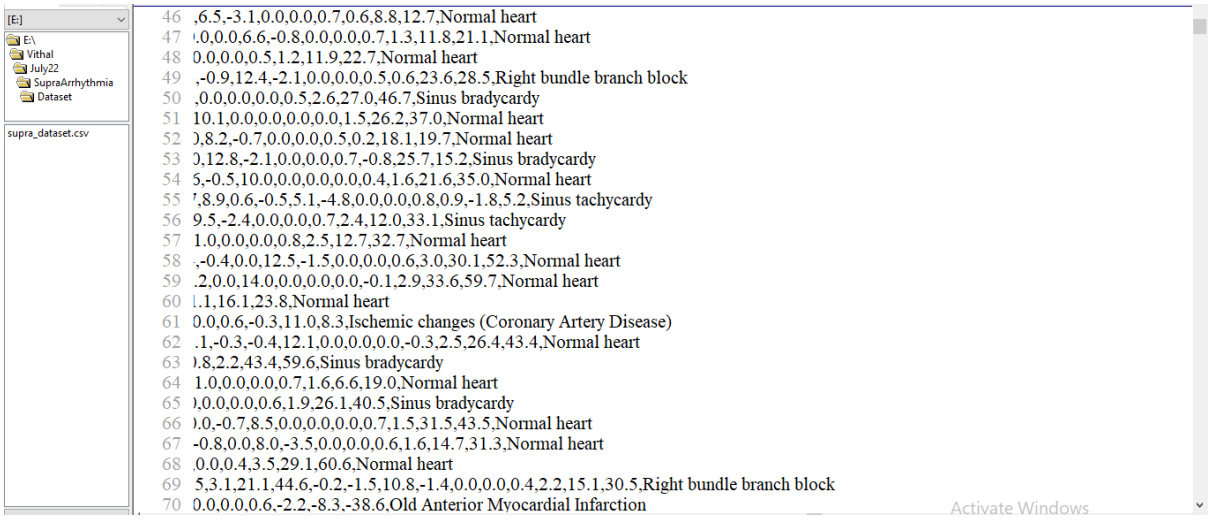


Figure 5.1. Sample dataset.

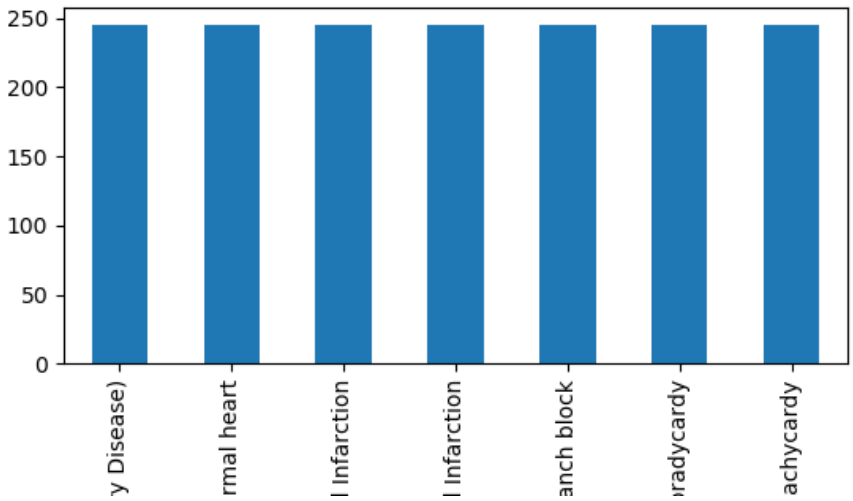


Figure 5.2. Records found for each class.

In Figure 5.2, we can see dataset loaded and in graph x-axis represents 7 different disease stages and y-axis represents number of records found for that disease in dataset.

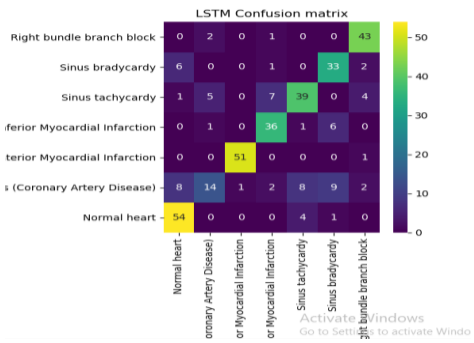


Figure 5.3. Confusion Matrix of LSTM.

In above screen with LSTM we got 78% accuracy and in confusion matrix graph x-axis represents Predicted classes and y-axis represents TRUE classes and all blue colour boxes count are wrong prediction and different colour boxes count are correct prediction and we can see LSTM predicted so many wrong classes.

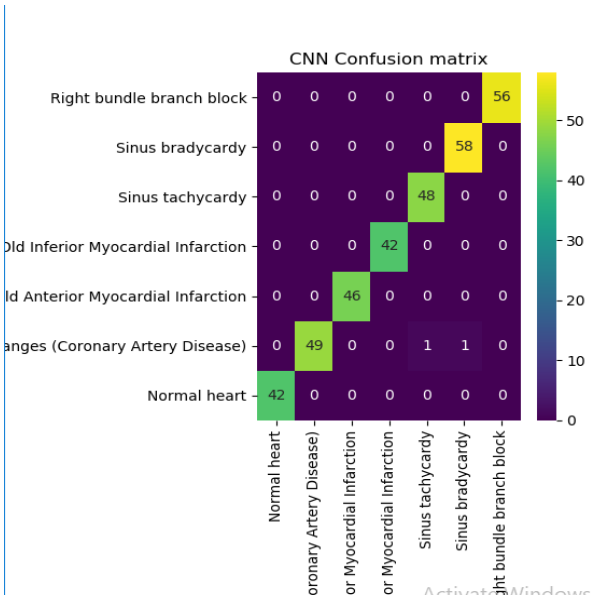


Figure 5.4. 1D-CNN Confusion matrix.

In above screen with CNN, we got 99% accuracy and in confusion matrix graph only 2 counts in blue colour boxes are wrong prediction and rest are correct prediction.

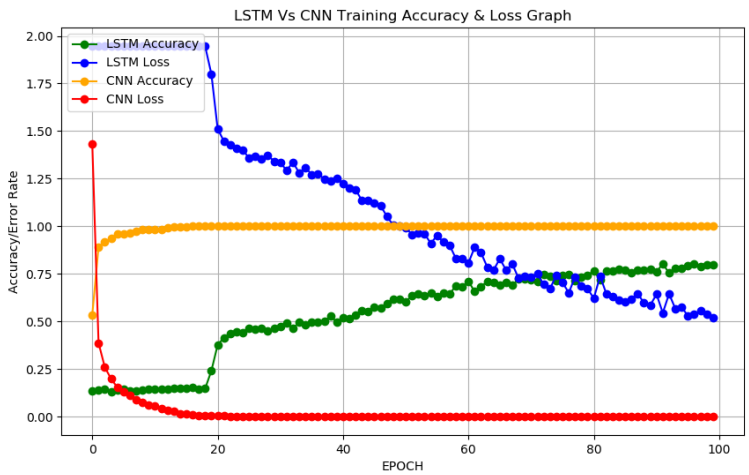


Figure 5.5. Accuracy and Loss performance graph.

In above graph x-axis represents training epoch and y-axis represents training accuracy and loss values and green colour line represents LSTM accuracy and orange colour line represents CNN accuracy and red colour line represents CNN loss and blue line represents LSTM loss and in above graph we can see both algorithms accuracy got increase in every epoch and loss get decrease.

Table 1. Performance comparison

Metric	Existing LSTM	Proposed 1D-CNN
Accuracy	83.38	99.54
Precision	83.075	99.24
Recall	83.0074	99.56
F-Score	82.32	99.24
Sensitivity	100	100
Specificity	91.66	97.28

CHAPTER 7
CONCLUSION

Cardiac arrhythmia is an irregularity in heart rhythm. Some types of cardiac arrhythmia can lead to complications like stroke, heart attack and may even lead to sudden cardiac death. So, timely detection and diagnosis of arrhythmia is especially important. Once arrhythmia is detected, next stage of identification of category of arrhythmia can be done. We developed an automated non-invasive system based on deep learning networks to perform the basic classification of a given ECG data as belonging to normal ECG or abnormal (having arrhythmia) ECG using the most popular public ally available MIT-BIH arrhythmia database.

We compared the performance using a variety of deep learning architectures of LSTM, 1D-CNN. With concern on computational cost, we are not able to train more complex architecture. The reported results can be further improved by using more complex deep learning architecture. The complex network architectures can be trained by using advanced hardware and following distributed approach in training that we are incompetent to try.

We have discussed the role of deep learning techniques such as 1D-CNN in the task of arrhythmia classification. The highlight of the proposed method is that it does not need any noise filtering and feature engineering mechanisms. The results obtained prove that the performance of our method is better than other published results in effectively classifying ECG as belonging to normal or arrhythmia class.

Future scope

Though deep learning networks produces excellent results, the disadvantage lies in the insufficient understanding of the complex inner mechanisms of the deep learning networks. This could be overcome by remodelling the nonlinear deep networks to a linear form by computing eigenvalues and eigenvectors in different time steps. The future work can be the collection of real-world datasets from hospitals having cardiac care units and the application of the same methodologies to the real datasets.

REFERENCES

[1] N Jafarnia, Dabanloo, G Attarodi. “A New Pattern Recognition Method for Detection and Localization of Myocardial Infarction Using T-Wave Integral and Total Integral as Extracted Features from One Cycle of ECG Signal”. 2014/01/01. Pp 818-824, vol.7, Journal of Biomedical Science and Engineering.

[2] L. N. Sharma, R. K. Tripathy, and S. Dandapat, "Multiscale Energy and Eigenspace Approach to Detection and Localization of Myocardial Infarction," in IEEE Transactions on Biomedical Engineering, vol. 62, no. 7, pp. 1827-1837, July 2015, doi: 10.1109/TBME.2015.2405134.

[3] U. R Acharya, H Fujita, Vidya K. Sudarshan, Shu LO, MD Adam, Joel E.W. Koh, JH Tan, Dhanjoo N. Ghista, Roshan Joy Martis, Chua K. Chua, Chua Kok Poo, Ru San Tan. “Automated detection and localization of myocardial infarction using electrocardiogram: a comparative study of different leads”, Knowledge-Based Systems, Volume 99, 2016, Pages 146-156, ISSN 0950-7051, https://doi.org/10.1016/j.knosys.2016.01.040.

[4] B. Mohammadzadeh Asl and S. K. Setarehdan, "Neural network-based arrhythmia classification using Heart Rate Variability signal," 2006 14th European Signal Processing Conference, 2006, pp. 1-4.

[5] . Vishwa-Abhinav, LM Dixit S, P Varadwaj. “Clasification Of Arrhythmic ECG Data Using Machine Learning Techniques”, International Jorunal of Interactive Multimedia and Artificial Intelligence, Pp 67-70, vol.1, 2011.

[6] G Swapna, Dhanjoo N. Ghista, RJ Martis, Alvin P. C. ANG, and SV Sree. “Ecg Signal Generation and Heart Rate Variability Signal Extraction: Signal Processing, Features Detection, And Their Correlation with Cardiac Diseases”, Journal of Mechanics in Medicine and Biology, Vol.12, 2012, https://doi.org/10.1142/S021951941240012X.

[7] V. G. Sujadevi, K. P. Soman, R. Vinayakumar, Real-Time Detection of Atrial Fibrillation from Short Time Single Lead ECG Traces Using Recurrent Neural Networks, Intelligent Systems Technologies, and Applications, 2018, Volume 683, ISBN: 978-3-319-68384-3.

[8] R. K. Pathinarupothi, R. Vinaykumar, E. Rangan, E. Gopalakrishnan and K. P. Soman, "Instantaneous heart rate as a robust feature for sleep apnea severity detection using deep learning," 2017 IEEE EMBS International Conference on Biomedical & Health Informatics (BHI), 2017, pp. 293-296, doi: 10.1109/BHI.2017.7897263.

[9] Goldberger AL, Amaral LA, Glass L, Hausdorff JM, Ivanov PC, Mark RG, Mietus JE, Moody GB, Peng CK, Stanley HE. Physio Bank, Physio Toolkit, and PhysioNet: components of a new research resource for complex physiologic signals. Circulation. 2000 Jun 13;101(23): E215-20. doi: 10.1161/01.cir.101.23. e215. PMID: 10851218.

[10] F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to forget continual prediction with LSTM," 1999 Ninth International Conference on Artificial Neural Networks ICANN 99. (Conf. Publ. No. 470), 1999, pp. 850-855 vol.2, doi: 10.1049/cp:19991218.

[11] Ian Goodfellow and Yoshua Bengio and Aaron Courville; Deep Learning; Book in preparation for MIT Press; 2016.

[12] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proceedings of the IEEE, 86(1998), 2278-2324.

[13] Y. Bengio, Learning deep architectures for AI, Foundations and Trends in Machine Learning, 2(2009) 1-127.

[14] Coşkun, M., Yildirim, Ö., Uçar, A., & Demir, Y. (2017). An Overview of Popular Deep Learning Methods. European Journal of Technique, 7(2), 165-176.

[15] Paweł Pławiak; Novel methodology of cardiac health recognition based on ECG signals and evolutionary-neural system; Expert Systems with Applications; 2018; 92; 334 – 349.

[16] Paweł Pławiak; Novel genetic ensembles of classifiers applied to myocardium dysfunction recognition based on ECG signals; Swarm and Evolutionary Computation; 2018; 39; 192 – 208.

[17] G. B. Moody and R. G. Mark; The impact of the MIT-BIH Arrhythmia Database; IEEE Engineering in Medicine and Biology Magazine; 2001; 20; 45-50

[18] Bengio, Y. (2009). Learning deep architectures for AI. Foundations and trends in Machine Learning, 2(1),1-127.

[19] Reza Moazzezi. (2011) “Change-based population coding.” PhD thesis, UCL (University College London.

APPENDIX- I
SOFTWARE ENVIRONMENT

What is Python?

Below are some facts about Python.

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
- Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just

means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners[1], Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

Python Development Steps

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it."Some changes in Python 7.3:

- Print is now a function.
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g., a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e., int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behaviour.
- Text Vs. Data Instead of Unicode Vs. 8-bit

Purpose

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project

TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and

hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here.The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

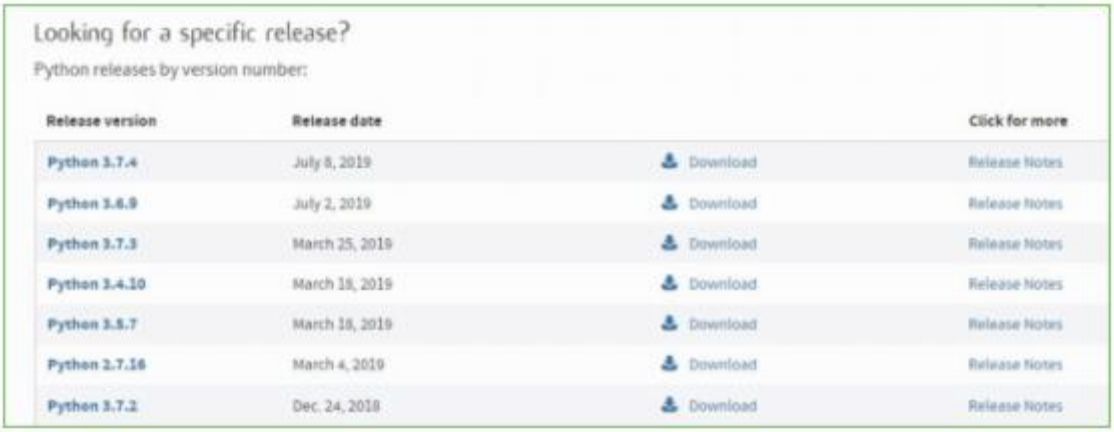
Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4



Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		68111671e5b2db4aef7b9ab01bf0f9be	23017663	SIG
XZ compressed source tarball	Source release		d33e4aae66097051c2eca45ee3604803	17131432	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583daff1a442cba8cee08e6	34898416	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a457738f5e4a936b241f	28082845	SIG
Windows help file	Windows		d639995733a2c96b2ac56cade6b4f7cd2	8131761	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c8cf8d9ec0b9abe83184a40726a2	7504391	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76debdb304a3a583e563e00	26480368	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1c608b6d73ae8e53a3bd351b4bd2	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3bd19b841879fda941335741398d	6741626	SIG
Windows x86 executable installer	Windows		33cc602942a54446a3d8451476394789	25663848	SIG
Windows x86 web-based installer	Windows		1b670cfa5d317df82c30983ea371d87c	1324608	SIG

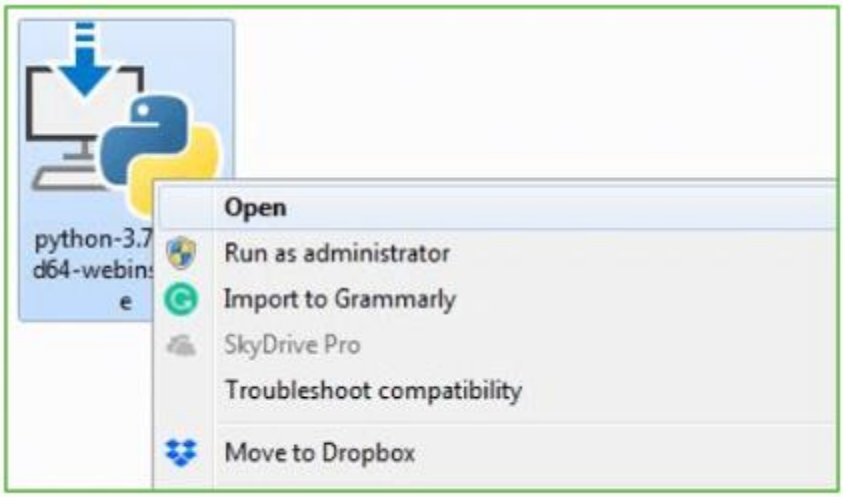
- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

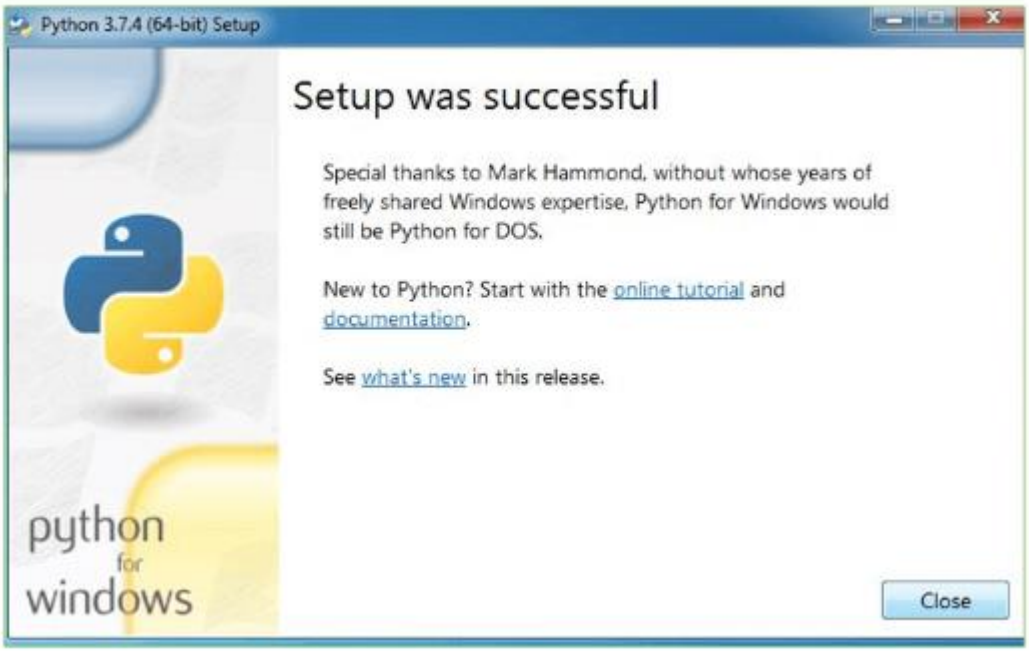
Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

Step 3: Click on Install NOW After the installation is successful. Click on Close.



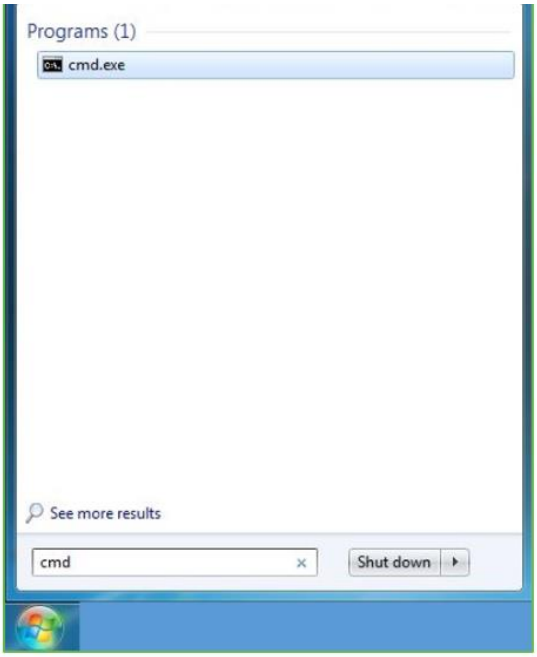
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

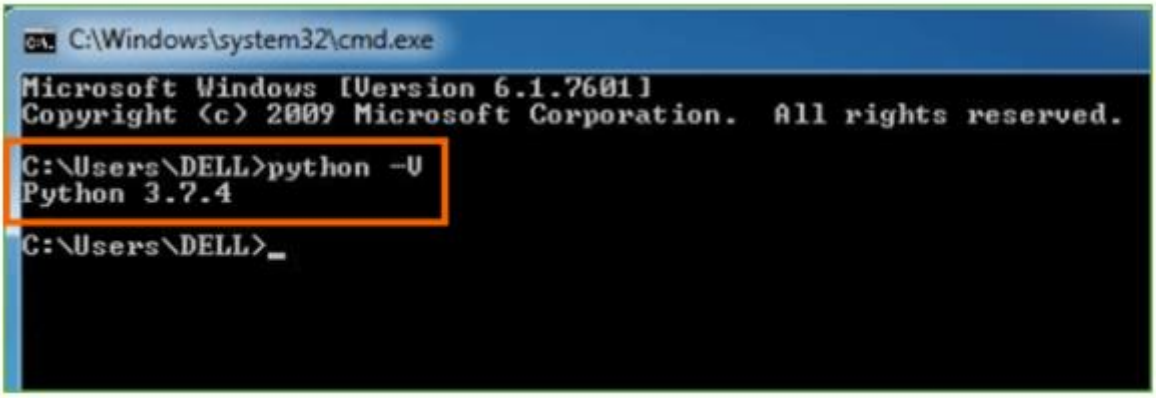
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python –V and press Enter.



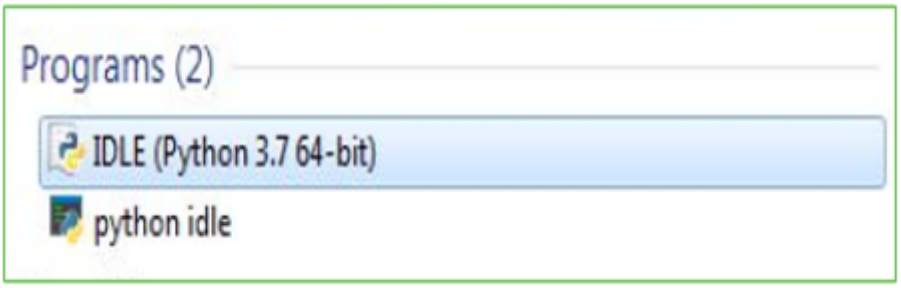
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

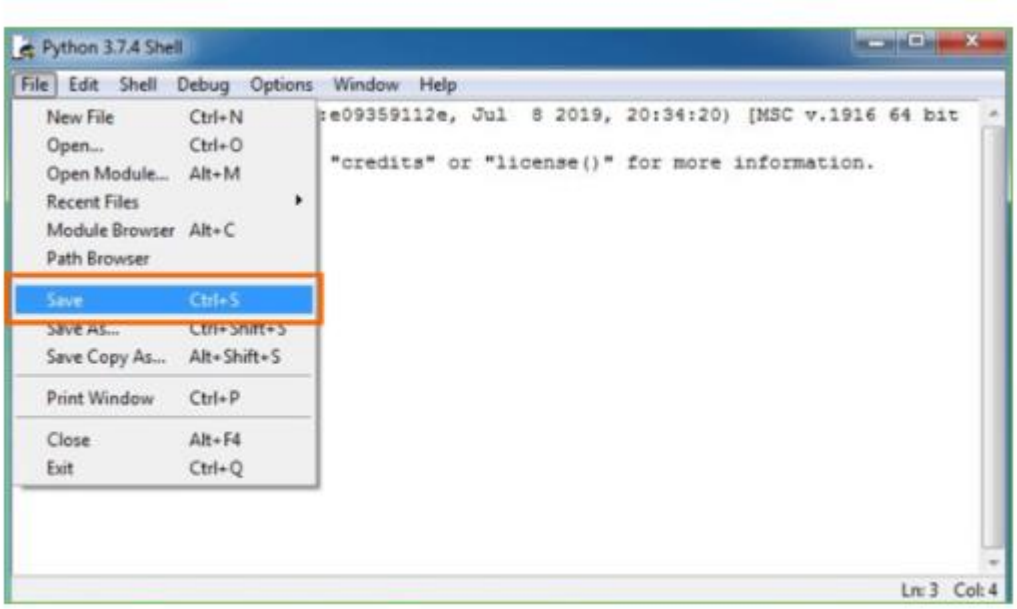
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



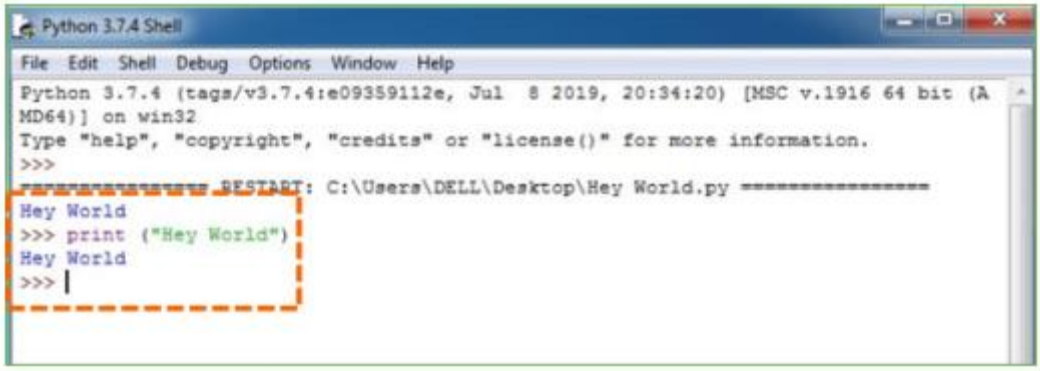
Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print (“Hey World”) and Press Enter.



You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.

Note: Unlike Java, Python does not need semicolons at the end of the statements otherwise it won't work.